Claude Certified Architect — Foundations

A First-Principles Study Guide

> *Built from the official exam guide and a 200-question practice bank. Ordered so each idea only uses ideas you've already met.*

How to read this guide

Every section follows the same shape:

1. **The first principle** — the underlying truth that makes the rule true.
2. **What follows from it** — the specific rules and patterns the exam tests.
3. **Concrete example(s)** — at least one.
4. **Anti-patterns** — what *looks* right but isn't, and why.

If you only remember one meta-principle, remember this:

> ***The exam rewards architectural reasoning, not vocabulary recall.*** *Every "right" answer addresses a root cause; every wrong answer treats a symptom. The exam writers signpost wrong answers with phrases like "the tempting shortcut", "a commonly proposed patch", "works in the easy case and leaves the hard cases silently broken". Train yourself to recognize that smell.*

PART 0 — Foundations (read this first)

These are the atoms. Everything else is composed of them.

0.1 What an LLM actually is, in one paragraph

An LLM (Large Language Model) is a function from **a sequence of tokens in** to **a probability distribution over the next token**. Sampling that distribution repeatedly produces the response. It has no memory between calls, no access to the world, no notion of time, no built-in tools. Each call is stateless: whatever the model "knows" about your conversation is whatever you put into the input. **This single fact is the source of more than half the rules in this exam.**

If the model "forgot" something — you didn't put it in the input. If the model "hallucinated" a fact — you didn't constrain it to a source. If the model "didn't follow your rule" — your rule lives in prose; the model is probabilistic.

0.2 Tokens and why context is finite

A **token** is a chunk of text (roughly ¾ of a word). The input + the output together must fit in the model's **context window** (e.g. 200K tokens). Every byte of system prompt, every tool description, every prior message, every tool result counts against the budget. So:

- Long tool outputs **steal capacity** from later reasoning.
- Multi-turn conversations grow until they push out the model's ability to attend to early facts.
- "Just use a bigger context window" is rarely the right answer — it changes the size of the room, not the **attention quality** inside it (see "lost in the middle", §6.1).

0.3 What a "tool" is

A **tool** is a function description (name, JSON-schema parameters, prose description) that you give the model. The model can choose to "call" the tool by emitting structured output that names the tool and supplies arguments. **Your code** then actually runs the function, captures the result, and feeds it back to the model as a message. The model never executes anything itself.

So a "tool call" is a request, not an action. You execute it. You return the result. The model decides what to do next.

0.4 The three layers you'll be tested on

Layer	What it is	What you build with it
Claude API	Raw HTTP endpoint: messages, tools, tool_choice, stop_reason	Anything custom
Claude Agent SDK	A library on top of the API that runs the agentic loop, manages subagents (`Task` tool), and supports hooks	Production agents (support bots, research pipelines)
Claude Code	A CLI/IDE app built on the Agent SDK, configured by `CLAUDE.md`, `.claude/`, MCP servers	Developer workflows
MCP (Model Context Protocol)	A standard for exposing tools and resources to a model from an external server	Backend integrations

These layers stack. Claude Code uses the Agent SDK. The Agent SDK uses the API. MCP is a way of *publishing* tools that any of those layers can consume.

0.5 The agentic loop — the most important concept on the exam

An "agent" is a `while` loop, not a magic pattern. The loop is:

```
loop:
    response = call_claude(messages, tools)
    if response.stop_reason == "end_turn":
        break  # the model is done, return text to user
    if response.stop_reason == "tool_use":
        for each tool_use in response:
            result = execute(tool_use)
            messages.append(tool_result)
        continue  # ask Claude what to do next, given the new tool results
```

That's it. Several exam questions hinge on getting this exactly right, so let's name the failure modes immediately:

- **Wrong: terminate when the response contains text.** The model may emit explanatory text *between* tool calls. Text doesn't mean "done."
- **Wrong: terminate after N iterations only.** You need a hard cap as a *safety rail*, but the *primary* termination signal is `stop_reason == "end_turn"`.
- **Wrong: parse natural language ("Done!" / "Finished") to decide.** Probabilistic and brittle.
- **Right:** branch on `stop_reason`. `"tool_use"` → execute and loop. `"end_turn"` → stop. `"max_tokens"` → handle (e.g. summarize tool output and continue, or surface the truncation).

0.6 The deepest principle in the entire exam

> ***Prose instructions to the model are probabilistic. Code that runs around the model is deterministic. When the cost of failure is high, push the rule into code.***

Every "should this be a prompt rule, a tool description, or a hook?" question reduces to this. The hierarchy of enforcement strength, weakest → strongest:

1. **System prompt rule** ("you must verify before refunding") — probabilistic. Failure rate > 0.
2. **Few-shot example** ("here's an example of doing it right") — probabilistic, slightly better.
3. **Tool description** ("only call this after X") — probabilistic, scoped to one tool.
4. **JSON schema constraint** ("this field is an enum of {A,B,C}") — eliminates a *class* of failures (invalid values).
5. **`tool_choice` configuration** ("the next reply must call a tool") — eliminates "no tool was called."
6. **PreToolUse / PostToolUse hook** — code that intercepts every relevant call and can block, transform, or normalize. **Deterministic.**
7. **Tool-layer logic** — the tool itself rejects invalid input or enforces idempotency.

When the question is "the agent does X 2-12% of the time even though we said not to," the answer is almost always to climb the ladder.

0.7 Why "use a bigger model" is almost never the right answer

The exam pattern: a wrong answer often offers "switch to a model with a larger context window" or "use a bigger/smarter model." This is wrong because:

- Bigger context windows don't fix **attention quality** (lost in the middle).
- Bigger models don't make probabilistic enforcement deterministic.
- Bigger models don't fix vague criteria, missing schemas, or stale tool results.

Treat "use a bigger model" the same way you treat "raise the temperature" or "add more emphasis to the prompt": almost always a distractor.

PART 1 — Domain 1: Agentic Architecture & Orchestration (27% of the exam)

This is the largest domain. It's about: building the loop correctly, and composing multiple agents safely.

1.1 The agentic loop in detail

Already introduced in §0.5. The exam tests three nuances repeatedly:

1.1.1 Stop reasons — exhaustive table

`stop_reason`	What it means	What your loop does
`tool_use`	The model wants to call one or more tools	Execute them, append `tool_result` messages, loop
`end_turn`	The model believes it is finished	Return text to user, exit loop
`max_tokens`	The model hit the token cap mid-response	**Handle explicitly** — usually summarize the partial output and continue, or surface the truncation. Don't treat as `end_turn`.

`stop_sequence`	Hit a configured stop string	Application-defined

The trap question: a tool returns a huge result. The next model call hits `max_tokens`. The loop has been written as `while stop_reason !== "end_turn"`, so it terminates silently with no user-facing response. Correct fix: **detect `max_tokens` and handle it**.

1.1.2 Iteration cap — the safety rail

Every loop needs a hard cap (e.g. 25 iterations). Pair the cap with a fallback (escalate to a human, return a partial result with a "could not converge" note). The cap is not the *primary* termination — `stop_reason` is. The cap is the airbag.

1.1.3 Tool errors are normal loop input

If a tool returns an error, the agent reads it as a `tool_result`, decides what to do (retry with different args, try another tool, give up and report), and the loop continues. **Tool errors should not terminate the loop.** They terminate the *tool call*.

1.2 Hooks — the deterministic enforcement layer

The Agent SDK exposes hooks:

- **`PreToolUse` hook** — runs before a tool call is executed. Can inspect the proposed call and **block** it, modify it, or let it through.
- **`PostToolUse` hook** — runs after a tool call returns. Can transform the result before the model sees it.

Hooks are ordinary code. They are deterministic.

1.2.1 What `PreToolUse` is for

- **Prerequisite gates.** Block `process_refund` until `get_customer` has completed in this session.
- **Policy enforcement.** Reject any `process_refund` call where `amount > 500`; redirect to escalation.
- **Confirmation walls.** Reject any money-moving call that doesn't have a corresponding user-confirmation token in the conversation.
- **Session-state checks.** "Did the search-tickets call run before create-ticket?"

1.2.2 What `PostToolUse` is for

- **Data normalization.** Three backends return timestamps in three formats; the hook rewrites all of them to ISO 8601 before the model sees them. (The model never has to convert, so it never miscompares.)
- **Output trimming.** A tool returns 60 fields when 4 matter; the hook drops the rest.
- **Audit logging.** Emit structured telemetry for every tool call.
- **Workflow chaining.** After every `Edit`, run the tests automatically.

1.2.3 Why this matters more than any other lesson

Many exam questions describe "we have a 2% failure rate and want it to be 0." If the rule is checkable from data the system already has (the call's arguments, the session history, the tool's response), the answer is **a hook**, not a stronger prompt. Prompts are probabilistic; hooks are not.

1.3 Multi-agent orchestration

A **coordinator** is a top-level agent whose tool set includes `Task` (which spawns subagents). **Subagents** are independent Claude conversations that run a specific specialist role (web search, document analysis, synthesis, report generation, etc.).

1.3.1 The two foundational facts about subagents

1. **Subagents do not inherit the coordinator's context.** They start with whatever you put in their prompt. If you spawn a "research healthcare AI" subagent and only give it the string "Research AI in healthcare", it has no idea what the user is actually after, what's been found so far, or what subsidiary questions remain. It will produce shallow generic output. **Fix: pass complete context (goal, scope, prior findings, expected output shape) explicitly in the subagent's prompt.**
2. **The coordinator must include `"Task"` in its `allowedTools`.** Without it, it cannot spawn subagents. (An invisible setup question.)

1.3.2 Parallel subagent spawning

To run subagents in parallel, the coordinator emits **multiple `Task` tool calls in a single response turn**. The Agent SDK runs them concurrently. To run them sequentially, emit one `Task` call per turn and wait. Sequential is correct only when there's a real data dependency (subagent B needs subagent A's output).

> *Trap: "How do I parallelize?" Wrong answers include `async: true` (fictional), `fork_session` (different purpose, see §1.6), middleware (over-engineered). Right answer:* ***multiple `Task` calls in one response.***

1.3.3 Coordinator dynamic routing

A coordinator that always invokes the full pipeline for every query is wasteful. A correctly-designed coordinator **assesses query complexity** and chooses which subagents to invoke. Simple factual lookup → maybe just web search. Deep research → full pipeline. (Match the workflow to the work.)

1.3.4 Coordinator decomposition is where coverage breaks

A common exam scenario: "The reports cover only X subset of the topic." Each subagent worked correctly within its assigned scope. The coordinator decomposed the topic too narrowly. **The fix is in the coordinator, not the subagents.** This is what the official sample question 7 is about.

A robust coordinator runs an **iterative refinement loop**: after synthesis, evaluate for coverage gaps; if there are gaps, re-delegate to search/analysis with targeted queries; re-synthesize. Repeat until coverage criteria are met.

1.3.5 Routing all communication through the coordinator

Subagents should not talk to each other directly. Everything routes through the coordinator. Why?

- **Observability.** One place to log, one place to debug.
- **Consistent error handling.** The coordinator is the single decision point for retries, fallbacks, partial-result acceptance.
- **Information control.** The coordinator decides what each subagent sees, preventing context bleed and unintended dependencies.

1.3.6 Tool count discipline (links to §2.3)

Each subagent gets only the tools it needs. The synthesis agent does not get web-search tools. (If it does, it'll wander outside its specialization and produce bad results.) Exception: a tightly-scoped helper for a high-frequency need is OK — e.g., a `verify_fact` tool for the synthesis agent, where 85% of verifications are simple lookups and only 15% need full delegation back through the coordinator.

1.4 Subagent context passing

Two facts:

- The subagent's prompt must contain everything it needs.
- For provenance, **pass structured data**, not prose. A finding should be `{claim, evidence_excerpt, source_url, publication_date}`, not "Smith said X."

1.4.1 Including findings from prior agents

If the synthesis subagent needs the web search results and document analysis output, **paste them into the synthesis subagent's prompt directly**. There's no automatic inheritance. (One of the sample exercises in the exam guide is exactly this.)

1.5 Multi-step workflow patterns

1.5.1 Programmatic prerequisites (the canonical example)

> *Production data: 8% of the time, the agent processes refunds without first verifying customer identity. What's the fix?*

- **Wrong: stronger prompt wording.** Probabilistic; 8% just becomes 6%.
- **Wrong: `tool_choice` forced selection on every turn.** Wrong granularity — the agent shouldn't be *forced* to verify, it should be *blocked* from refunding without verifying.
- **Wrong: routing classifier.** Solves a different problem (which tools to expose), not the ordering problem.
- **Right: A `PreToolUse` hook on `process_refund` that fails the call if no verified `get_customer` has completed in this session.** Deterministic. The agent is free to call refunds; it just *can't* before customer is verified.

1.5.2 Structured handoffs to humans

When the agent escalates, the human reviewer cannot see the conversation. They need a **structured summary**: customer ID, root cause, recommended action, refund amount (if any). Not a full transcript (too long, no synthesis), not just an email address (loses what was concluded).

The handoff *quality* (did it include those fields?) is what predicts how fast the human resolves the case. That's the right metric for hand-off success.

1.5.3 Decomposing multi-concern requests

A user reports three issues in one message. Wrong: handle them one at a time in series, mixing context. Right: decompose into distinct items, investigate in parallel using shared context, synthesize one unified resolution.

1.6 Session management

1.6.1 The four levers

- `--resume <session-name>` — continue a specific named session. Use when prior context is mostly still valid.
- `fork_session` — branch independently from a shared baseline. Use to compare two approaches starting from the same analysis.
- `/compact` — compress the current session in place. Use when **context is large but facts are still valid** (no code changes, no stale state).

- **Start fresh + inject summary** — abandon the session entirely. Use when **prior tool results are stale** (code has changed, world has moved). Resuming with stale tool results just propagates the staleness.

1.6.2 The trap question

> *You finished a long investigation yesterday. Today the repo has significant changes. You resume — the agent references functions that no longer exist. Best next step?*

- `/compact` is wrong: it compresses the staleness, doesn't remove it.
- Re-reading every file is wrong: doesn't address the structural staleness in the model's prior reasoning.
- A "code may have changed" prompt reminder is wrong: a probabilistic patch on a structural problem.
- **Right: start a new session with a structured summary of prior findings.** Throw away the stale tool results.

1.6.3 `fork_session` use cases

Three siblings of the same baseline:

- "Compare two refactoring strategies starting from the same analysis."
- "Try three variations of a migration."
- "A/B test two prompt approaches against the same context."

1.7 Error recovery in multi-agent systems

If a subagent fails, the coordinator should:

- Receive a **structured error** (failure type, what was attempted, partial results, alternatives) — **not** a generic "search unavailable."
- Decide: retry with different args, try alternative subagent, accept partial results.
- **Annotate the final output** with which sections are well-covered vs which have gaps due to unavailable sources.

Wrong patterns:

- Crash the whole pipeline on one failure.
- Silently return empty results pretending it was a success.
- Retry all 8 subagents when only 3 failed (re-runs successful work).

When 3 of 8 parallel subagents fail, **track success per-subagent (using `custom_id`-style identifiers) and retry only the failures with backoff.**

1.8 Idempotency and two-phase patterns

Some actions are irreversible (cancel subscription, send email, process refund). The agent occasionally repeats them when the user rephrases ("yes, refund it" / "please go ahead with the refund").

- **Idempotency keys** at the tool layer prevent duplicate execution.
- **Two-phase tools**: `preview_cancel` returns the would-be effect and a confirmation token; `confirm_cancel` requires that token. Structurally, you can't double-fire.

These are tool-design fixes, not prompt fixes.

1.9 The Explore subagent

Claude Code ships with an Explore subagent for verbose discovery (mapping a large codebase, tracing dependencies). It runs in isolation and **returns a summary**. Anti-pattern: have Explore return a 4000-line listing into

the main session — that's worse than not using Explore at all. **Always prompt the subagent to summarize.**

1.10 Confidence is uncalibrated

LLM-generated confidence scores are correlated with verbosity, not correctness. Don't gate decisions on a `confidence` field the model emits about itself.

Use **structural checks** instead: does the math sum? Does the source URL exist? Does the schema validate? Did the prerequisite step complete? Those are deterministic.

1.11 Recurring Domain 1 anti-patterns to internalize

- "Add stronger language to the system prompt" — prose patch on a structural problem.
- "Add few-shot examples" — useful for *ambiguity*, not for *enforcement*.
- "Lower the temperature" — addresses creative drift, not loop logic, not enforcement, not coverage.
- "Use a bigger context window / smarter model" — doesn't fix attention quality, doesn't fix decomposition, doesn't fix prompt-vs-hook decisions.
- "Tell the agent to be careful" — never an answer.

PART 2 — Domain 2: Tool Design & MCP Integration (18% of the exam)

2.1 What MCP is, in one paragraph

Model Context Protocol is a standard for exposing two things to a model from an external server: **tools** (actions the model can call) and **resources** (passive content the model can see, like a schema or a catalog). MCP servers run as local processes or remote services. Claude Code and the Agent SDK can connect to MCP servers and discover their tools/resources at startup. The model then sees those tools alongside built-in ones.

The point of MCP is **standardization**: instead of writing a custom integration for every backend, you implement the MCP protocol once and any MCP-aware client can use it.

2.2 Tool descriptions are how the model picks the tool

The model selects tools based on (in order of importance):

1. The tool's **description**.
2. The tool's **name**.
3. The tool's **parameter schema**.
4. Examples in the system prompt.

Two tools with descriptions like "Retrieves customer information" and "Retrieves order details" will be confused constantly. The fix is **specificity**:

> "*Fetches a customer's billing history for the last 90 days. Use when the user asks about past charges or recent invoices. Input:* `customer_id` *(format:* `C-NNNNN`*). Returns billing records sorted newest-first.*"

A good description specifies **what, when, why, expected inputs, expected outputs**. Self-promotion ("USE THIS TOOL FOR EVERYTHING") distorts selection — it doesn't help.

2.3 The tool count rule

An agent with 4–5 well-described, role-relevant tools selects reliably. An agent with 18 tools selects unreliably. Decision complexity scales worse than linearly. **Trim aggressively.** This is the canonical question on tool count (D2-001).

2.4 Splitting generic tools

A single `analyze_document` tool used for extraction, summarization, and verification will be misused. Split it:

- `extract_data_points`
- `summarize_content`
- `verify_claim_against_source`

Each has a single purpose, a clean schema, an unambiguous description. The agent can pick correctly because the tools are no longer overlapping.

2.5 Naming and disambiguation

If two MCP servers both expose a `search` tool, namespace or rename them. The model sees them as "search" and "search" — collision is structural, not a description problem.

2.6 Structured error responses

Every MCP tool error response should include:

- **`isError: true`** (the MCP standard flag)
- **`errorCategory`** : one of `transient | validation | permission | business`
- **`isRetryable`** : boolean (explicit, not inferred)
- **Human-readable message**

Why: the agent uses this metadata to decide what to do.

- `transient` + `isRetryable: true` → the agent retries (maybe with backoff)
- `validation` → the agent fixes the input
- `business` (e.g. policy violation, refund-too-large) → the agent explains to the user and escalates
- `permission` + `isRetryable: false` → the agent escalates without retrying

A generic `{"status": "error", "message": "Operation failed"}` is the canonical bad response — the agent has no information to recover with.

2.7 Empty results vs failures

If a tool returns `[]` for both "no matches" and "upstream is down", you have lost a critical distinction. The agent will treat a failure as "successfully found nothing." **Always return a structured error for failures, distinct from a valid empty result.**

2.8 `tool_choice`

Three values:

Value	Meaning

`"auto"` (default)	Model may call a tool **or** return text
`"any"`	Model **must** call some tool, but chooses which
`{"type": "tool", "name": "X"}`	Model **must** call tool X specifically

There is no `"required"` value. (A trap distractor.)

When to use which:

- `"auto"` for normal conversational flow.
- `"any"` when the next response should always be a tool call (e.g. extraction pipeline that should never produce conversational text).
- Forced specific tool when you know which extraction schema applies (e.g. you've already classified the document as an invoice).

Important: `"any"` doesn't fix bad descriptions. The model still picks based on names and descriptions; it just *must* pick.

2.9 MCP server scoping

Two locations:

- `.mcp.json` at the project root — **shared via version control**. For team-wide MCP servers (the company's GitHub server, the shared Jira server).
- `~/.claude.json` in the user's home — **personal**. For experimental servers and personal preferences.

Don't commit personal experimental tools to `.mcp.json` — they pollute the team's tool catalog and increase selection ambiguity for everyone.

2.10 Secrets via env var expansion

`.mcp.json` supports `${ENV_VAR}` expansion. So:

```
{
  "github": {
    "command": "github-mcp",
    "env": {
      "GITHUB_TOKEN": "${GITHUB_TOKEN}"
    }
  }
}
```

The file commits cleanly; the secret lives in each developer's environment.

2.11 Resources vs tools

	Tool	Resource
Purpose	An **action** the model takes	**Ambient content** the model can see
Examples	`process_refund`, `search_web`, `read_file`	"list of available database tables", "documentation hierarchy", "issue summaries"

If the agent currently calls `list_topics` at the start of every session just to discover what's available — that's a sign the data should be a **resource**, not a tool. Resources eliminate exploratory tool calls.

2.12 Built-in tools (Claude Code)

Tool	Purpose
Read	Load a full file's contents
Write	Create/overwrite a file
Edit	Targeted modification using exact-text matching for an anchor
Bash	Run a shell command
Grep	Search file *contents* for patterns (function names, error strings)
Glob	Search file *paths* by pattern (`**/*.test.tsx`)

Common selection mistakes:

- Reading every file to "find" something → should be Grep.
- Listing files to find ones matching a pattern → should be Glob.
- Trying to Edit when the anchor text isn't unique → fall back to Read + Write.

The right way to explore an unfamiliar codebase is **incremental**: Grep for entry points, Read to follow imports, trace flows from there. **Never load all files upfront.**

2.13 Other tool-design rules from the bank

- **Validate inputs strictly at the schema level.** If a `customer_id` must match `C-\d{5}`, validate it. Reject mismatches with a structured error explaining the format. This stops the agent from passing names where IDs are expected.
- **Prefer optional/nullable fields over forced strings.** A required `email` field forces fabrication when the email isn't known. Make it nullable so the agent can ask the user instead.
- **Replace generic `run_shell` with purpose-specific tools** like `run_tests`, `run_linter`, `format_file`. Reduces blast radius without losing useful capability.
- **Restrict `http_fetch`-style tools** to host allowlists; otherwise SSRF risk.
- **Versioned tool names** for breaking changes; keep additive changes (new optional params) under the same name.
- **Progressive disclosure**: if a tool has a huge response, return a `{summary, reference_id}` and provide a companion `get_full(reference_id)` tool the agent can call when needed.
- **Sync-fast / async-slow split** for tools whose backend latency varies wildly: `do_thing` (returns fast or fails) and `do_thing_async` (returns a job ID; companion `check_status` tool).
- **Cost hints** belong in the **tool description**, not the system prompt — that way they apply wherever the tool is exposed.
- **Build vs buy**: prefer existing community MCP servers for standard integrations (GitHub, Jira). Build custom only for team-specific workflows.
- **Trim tool outputs at source** — adding a `fields: [...]` param or providing a slim variant (`get_customer_summary`) is cheaper than every agent post-filtering.

PART 3 — Domain 3: Claude Code Configuration & Workflows (20% of the exam)

This is your weakest domain from the practice exam. Slow down on every Domain 3 question.

3.1 The CLAUDE.md hierarchy

Three levels exist, and they **all stack** (none overrides another):

Location	Scope	Use for
`~/.claude/CLAUDE.md`	**User** — only this person, never shared via git	Personal preferences (e.g., "I like verbose explanations")
`<project-root>/CLAUDE.md` or `.claude/CLAUDE.md`	**Project** — shared via version control	Team coding standards, project-wide conventions
`<subdir>/CLAUDE.md`	**Directory** — applies when editing files in this subdirectory	Directory-specific patterns (e.g. `frontend/CLAUDE.md`)

Stacking, not overriding. When Claude edits a file in `src/components/`, it loads root + `src/components/CLAUDE.md` + user-level. They compose. If they conflict, resolve in content; don't expect one to suppress another.

3.1.1 The classic hierarchy mistake (D3-001)

A new team member doesn't get the team's standards. Investigation reveals the standards live in `~/.claude/CLAUDE.md` on the existing developers' machines. **Fix: move them to `<project>/CLAUDE.md` so they ship with the repo.** User-level configs are not shared.

3.1.2 The `/memory` command

Use `/memory` to verify which memory files are currently loaded. Diagnostic tool when behavior is inconsistent.

3.1.3 When to split CLAUDE.md

A focused 200-line CLAUDE.md is fine. **Split when it gets large and multi-topic** (e.g. 900+ lines covering API conventions, testing, database patterns, deployment). Split into `.claude/rules/<topic>.md` and reference them from CLAUDE.md via `@import`.

Don't preemptively modularize — complexity should justify the split.

3.2 `.claude/rules/` with glob paths

Rule files in `.claude/rules/` support YAML frontmatter:

```
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---
# Testing conventions
- Use `describe`/`it` blocks
```

```
- Use `userEvent` not `fireEvent`
- ...
```

The rule **only loads when Claude is editing a matching file**. This is the right tool when:

- A convention spans multiple directories (test files everywhere in the codebase)
- Rules apply by file *type*, not by directory
- You want to keep token usage low by not loading irrelevant rules

When to use rules vs subdirectory CLAUDE.md:

- **Rules with glob paths** → conventions tied to file *type/pattern*, spread across directories
- **Subdirectory CLAUDE.md** → conventions tied to *one specific directory subtree*

If your test files are scattered (e.g. `Button.test.tsx` next to `Button.tsx`), a subdirectory CLAUDE.md cannot reach them. A rules file with `paths: ["**/*.test.*"]` does.

3.3 `@import` for modular CLAUDE.md

```
# Project standards

@import .claude/rules/testing.md
@import .claude/rules/api-conventions.md
@import .claude/rules/deployment.md
```

The root file stays a scannable index; detail lives alongside its topic.

3.4 Slash commands vs skills

Both are reusable on-demand workflows, both are in `.claude/`, both can be project-scoped (committed) or user-scoped.

	Slash command	Skill
Location	`.claude/commands/` (project) or `~/.claude/commands/` (user)	`.claude/skills/` (project) or `~/.claude/skills/` (user)
File	A markdown file with the prompt template	A `SKILL.md` file with frontmatter + body
Frontmatter	Minimal	`context: fork`, `allowed-tools`, `argument-hint`
Best for	Quick reusable prompts (`/review`, `/impact`)	Multi-step workflows with tool-access discipline

Both are **invoked manually**. Universally needed standards should be in CLAUDE.md (always loaded), not converted to commands (must be invoked).

3.4.1 SKILL.md frontmatter — the three knobs you need to know

```
---
context: fork
allowed-tools: ["Read", "Grep", "Glob"]
```

```
argument-hint: "path to analyze"
---
```

Knob	Effect
`context: fork`	Run the skill in an **isolated sub-agent**. Verbose output stays inside the fork; only the final return value reaches the main session. Use for skills that produce hundreds of lines of intermediate work.
`allowed-tools`	Restrict which tools the skill may use. Set to `["Read", "Grep", "Glob"]` for read-only safety; omit dangerous tools (Bash, Write) when not needed.
`argument-hint`	Prompts the user for input when the skill is invoked without arguments. **No `require_args` or `interactive: true` knob exists** — `argument-hint` is the answer.

These three knobs are heavily tested. Memorize what each does and what each *does not* do.

3.5 Plan mode vs direct execution

	Plan mode	Direct execution
For	Complex tasks, architectural decisions, multiple valid approaches, multi-file changes	Well-scoped, well-understood single changes
Examples	Microservices restructuring, library migration touching 45+ files, choosing between integration approaches	Add a date validation conditional, fix a single-file bug with a clear stack trace
Risk it prevents	Costly rework when dependencies/decisions surface late	(Overhead avoided)

Plan mode is earned by ambiguity, not by caution. "Always use plan mode" is wrong because it adds review-round overhead to changes that have no open decisions.

Plan + direct can be combined: plan the migration, then execute the planned approach.

3.6 The Explore subagent

For verbose discovery phases (mapping a 200K-line codebase). Runs in isolation, returns summaries to the main session. Prevents context window exhaustion. Use **before** main investigation, not after; once the noise is in your main session, it's already done damage.

3.7 Iterative refinement techniques

When the model's first attempt isn't right, the bank tests three patterns:

- **Concrete input/output examples** — most effective when prose descriptions are interpreted inconsistently. 2–3 examples beats elaborate prose.
- **Test-driven iteration** — write tests first; iterate by sharing failures.
- **The interview pattern** — have Claude ask you questions before implementing in unfamiliar domains, to surface considerations you may not have anticipated (cache invalidation, failure modes, etc.).

- **One message vs sequential** — for **interacting** problems, one detailed message; for **independent** problems, sequential iteration.

3.8 Claude Code in CI/CD

The CI flag set:

Flag	Purpose
`-p` or `--print`	Non-interactive mode. Process, output, exit. **Without this, the job hangs** waiting for input.
`--output-format json`	Output as JSON instead of prose
`--json-schema <file>`	Enforce a specific JSON schema on the output

Usage:

```
claude -p --output-format json --json-schema ./review-schema.json "review this PR"
```

Claude Code uses **standard POSIX exit codes** — non-zero on failure. CI checks the exit code directly.

Fictional flags that are exam distractors: `--batch`, `--json-only`, `CLAUDE_HEADLESS` env var.

3.8.1 Two outputs from one call

If you want both machine-parseable JSON and a human summary, **don't run twice**. Run once with the JSON schema, then render the summary from the structured JSON in your CI script.

3.8.2 Same Claude session that wrote code shouldn't review it

Self-review is weak — the model retains its own reasoning context and is biased toward agreeing with itself. Use a **separate Claude instance** without prior context to review.

3.8.3 Incremental reviews

After the first PR review, for subsequent pushes, **include prior findings in the context** and instruct Claude to report only new or still-unaddressed issues. Otherwise developers get duplicate comments.

3.8.4 Test generation context

Provide existing test files in context so generation doesn't suggest duplicate scenarios. Document testing standards/fixtures in CLAUDE.md.

3.9 Settings-level tool restriction

If a tool (e.g. Bash) must be off-limits in a particular repo, that restriction lives in **settings-level `allowed-tools` permission config**, not in a CLAUDE.md instruction. The structural restriction means the tool isn't even visible to the model — there's nothing for it to skip.

PART 4 — Domain 4: Prompt Engineering & Structured Output (20% of the exam)

4.1 Specific criteria beat vague qualifiers

Wrong: "be conservative", "only report high-confidence findings", "be thorough", "be careful." Right: "flag comments only when claimed behavior contradicts actual code behavior." Right: "report bugs, security issues, and breaking API changes; skip minor style and local pattern preferences." Right: severity levels defined with **concrete code examples** for each level.

The deepest wrong-answer pattern: "rephrase the system prompt with stronger language." Strengthening prose doesn't fix vague criteria; rewriting with concrete anchors does.

4.1.1 The trust-decay problem

If a category of findings has 70% false positives, developers stop reading the entire output. **Temporarily disable the bad category** while you rewrite its prompt with explicit criteria. Trust is asymmetric: easier to lose than to rebuild.

4.2 Few-shot prompting

Few-shot examples are most effective when:

- The task involves **ambiguity** (which tool to pick when the request is borderline)
- The desired **output format** is specific (location, issue, severity, suggested fix)
- The input has **structural variety** (academic papers in inline-citation format vs numbered-references format)
- You want to **distinguish acceptable patterns from genuine issues** (false positive reduction)

Diversity beats volume. After 12 examples covering one pattern, adding more of the same pattern doesn't help. Diversifying the examples to cover different structural patterns does.

Few-shot is **not** the right tool for enforcement (climb to hooks for that).

4.3 Structured output via tool use + JSON schemas

The most reliable way to get schema-compliant output is `tool_use` **with a JSON schema**. The model is forced to produce arguments that satisfy the schema. JSON syntax errors disappear. Disallowed fields are impossible. Enums are constrained.

Anti-pattern: prompt-based JSON ("return JSON only, no markdown"). Always probabilistic. Code fences, prefatory prose, missing brackets — all possible.

Anti-pattern: regex-extract a JSON block from prose. Cat-and-mouse.

```
# Right: tool_use with schema
tools = [{
    "name": "extract_invoice",
    "input_schema": {
        "type": "object",
        "properties": {
            "vendor": {"type": "string"},
            "total": {"type": "number"},
```

```
            "line_items": {"type": "array", "minItems": 1, ...},
            ...
        },
        "required": ["vendor", "total", "line_items"]
    }
}]
```

4.3.1 What schemas eliminate vs don't

Schemas eliminate	Schemas don't eliminate
JSON syntax errors	Semantic errors (line items don't sum to stated total)
Disallowed fields	Wrong field placement
Invalid enum values	Wrong-value-in-right-shape
Wrong field types	Hallucinated content

For semantic checks, add **paired fields**: `stated_total` + `calculated_total`, then compare programmatically. Or `stated_value` + `verbatim_quote_supporting_value`. Self-consistency through structure.

4.4 Schema design

4.4.1 Required vs optional vs nullable

If a field **may not exist in the source document**, mark it nullable/optional. **A required-without-null field forces the model to fabricate.** This is the "fax number" pattern (D4-003) and the "email" pattern (D4-021).

4.4.2 The "other" + detail pattern

Enums that may need to grow: `category: enum["A", "B", "C", "other"]` plus `other_category: string | null`. When the data doesn't fit, the model picks `other` and describes it. Without this, the model is forced to misclassify.

4.4.3 First-class "unknown"

If "I can't tell from the source" is a valid answer, make it expressible:

```
{
  "conclusion": {
    "status": "supported" | "refuted" | "insufficient_evidence",
    "details": "..."
  }
}
```

Magic strings ("UNKNOWN", "N/A", "various") are brittle — they pass schema validation as ordinary strings but break downstream. Make uncertainty structural.

4.4.4 Presence vs value

If a field is sometimes present-but-zero and sometimes absent, **two fields**: `tip_present: boolean` and `tip_amount: number | null`. Don't conflate "0 tip" with "no tip field."

4.4.5 Right types for right things

Money: `number`, with a separate `currency` field. Not strings like `"$1,234.56"`. Dates: include a `publication_date` or `as_of_date` field on every data point that can change over time. (Otherwise temporal differences look like contradictions.)

4.4.6 minItems and other JSON Schema constraints

Yes, JSON Schema can enforce "non-empty array" via `minItems: 1`. Use schema constraints for simple structural rules; use post-extraction validation for cross-field rules.

4.5 Validation-retry loops

When the extraction validates as JSON but fails semantic checks (math doesn't add up, internal inconsistency), retry with the **failed extraction and the specific validation error appended to the prompt**:

> "*Your previous extraction had `stated_total: 100, calculated_total: 95`. The line items must sum to the stated total. Please re-extract from this document.*"

Critical limit: **retries are only useful for format/structural errors, not for missing information.** If the data simply isn't in the source, retrying won't conjure it. The right response is to mark the field null and surface the gap.

4.6 Confidence scores are uncalibrated

The bank repeatedly tests the principle: **don't gate decisions on a model-reported confidence number.** Confidence is correlated with verbosity, not accuracy.

Use **structural checks** for routing: schema validation, math consistency, source attribution presence, ambiguous/contradictory source flag.

For human-review routing, calibrate field-level confidence using a **labeled validation set** before trusting it.

4.7 The Message Batches API

Property	Value
Cost	**50% savings** vs synchronous API
Latency	Up to **24-hour** processing window, no SLA
Tool calling	**Does not support multi-turn tool calling** in a single request
Correlation	`custom_id` field on each request; carried back on the response

When to use:

- Overnight reports
- Weekly audits
- Nightly test generation
- Any non-blocking, latency-tolerant workload

When **not** to use:

- **Blocking pre-merge checks** (developers waiting for the result) — the 24h tail is unacceptable
- Workflows requiring multi-turn tool calling in a single request

4.7.1 SLA arithmetic

If you need a 30-hour SLA and the batch API can take 24 hours, you can submit on **4-hour windows** (24h batch + 4h scheduling buffer = 28h, with margin).

4.7.2 Batch failure handling

Use `custom_id` to identify each request. On batch completion, **resubmit only failed documents** (with modifications: e.g. chunking documents that exceeded context limits). Don't rerun the whole batch.

Use prompt refinement on a **sample** before batch-processing 10,000 documents. Iterating on a 100-doc sample is cheap; iterating on the full batch is expensive.

4.8 Multi-instance and multi-pass review

4.8.1 Self-review limitations

A model that just generated code retains its reasoning context — it's less likely to question its own decisions in the same session. **Use a separate Claude instance** for review.

4.8.2 Multi-pass for large reviews

For PRs spanning many files: **per-file passes** for local issues + **separate cross-file integration pass** for data flow. Avoids attention dilution and contradictory findings (where the same pattern is flagged in one file and approved in another).

Wrong patterns:

- "Use a bigger context window" — doesn't fix attention quality
- "Ask developers to split PRs" — shifts burden, doesn't fix the system
- "Run three passes and only flag agreement" — would suppress real bugs caught intermittently

4.9 Other prompt engineering bits

- **Stable, task-independent instructions go in the system prompt.** Per-task specifics go in the user message.
- **Prefill the assistant turn** with `<thinking>` tags (or use extended thinking) when you want guaranteed step-by-step reasoning.
- **Trim a prompt by removing**: restated rules, irrelevant background, filler ("please be accurate"). **Keep**: criteria, examples, schema constraints.
- **Temperature 0** reduces variance but doesn't fix undefined criteria. Treat it as a side knob, not a reliability solution.
- **Seed + temp 0** further reduces variance but isn't a strict guarantee across model versions.
- **Extended thinking** trades latency for reasoning depth. Enable for hard reasoning tasks; leave off for structured extraction and simple classification where it doesn't help.
- **Prompt caching** dramatically reduces cost/latency when a stable 5K+ token prefix is shared across many calls. Always use it for that pattern.

PART 5 — Domain 5: Context Management & Reliability (15% of the exam)

This domain is small but has high-leverage patterns that apply everywhere.

5.1 The lost-in-the-middle effect

Models reliably attend to the **beginning** and **end** of long inputs and may underweight the middle. So:

- Place **key findings summaries at the beginning** of aggregated inputs.
- Use **explicit section headers** to aid retrieval.
- Put the **most important items at the edges** when you can.
- For very long sources (100+ page PDFs): **chunk and process in focused passes**.

Wrong patterns: "Use a larger context window" (worsens the curve shape), "increase max_tokens" (changes output, not input attention), "paste the whole PDF twice."

5.2 Tool output trimming at source

When a tool returns 60 fields and only 4 matter, after a few calls the context is bloated and the model misses facts. **The fix is at the tool**: add a `fields: [...]` parameter, or provide a slim `get_customer_summary` variant. Source-side trimming is the most efficient context management.

If the tool can't be changed, use a **`PostToolUse` hook** to trim the response before the model sees it.

5.3 The "case facts" block pattern

For long, multi-issue conversations: **extract transactional facts** (amounts, dates, order IDs, statuses) into a structured "case facts" block that gets injected into **every prompt**, outside the summarized history. Summary compresses prose; the case-facts block keeps numbers literal.

This prevents the canonical failure mode where the agent confuses $47.99 with $23.50 after summarization.

5.4 Scratchpad files for long sessions

For exploration sessions: have the agent record key findings in a scratchpad file as it goes, then reference the file for subsequent questions. **The scratchpad survives summarization.**

The agent stops re-reading files because it can re-read its own notes instead.

For multi-agent systems with crash recovery: **structured agent state exports** (manifests) that the coordinator loads on resume and injects into agent prompts.

5.5 Provenance: claim-source mappings

If you don't preserve source attribution **at every step**, summarization destroys it. The downstream report looks authoritative but isn't auditable.

The structural fix:

- Every subagent's output schema requires `source_url`, `document_name`, and `relevant_excerpt` per claim.
- The synthesis agent's output schema requires the same — sources must propagate.
- **Required, not nullable.** If a claim can't be sourced, it shouldn't be emitted as a claim.

To prevent placeholder sources like "various", **narrow the source field type** to a structured object `{name, url, publication_date}` with format constraints (URL pattern). "various" can't validate.

For numeric claims, require `verbatim_quote` from the source supporting the number. Empty quote → schema violation. Forces the model to actually find the source rather than fabricate.

5.6 Conflicting source data

Two credible sources disagree (one says 30%, one says 45%). Wrong:

- Average them.
- Pick the more recent one and discard the other.
- Pick silently.

Right: **preserve both values with source attribution and explicitly annotate the conflict.** Let downstream decide. The synthesis output should structurally distinguish **established findings** from **contested findings**.

When a single document has internal conflicts (abstract says X, figure says Y), the analyzing subagent **completes the analysis with both values included**, annotated as conflicting. Don't escalate; don't pick. The coordinator decides reconciliation later.

5.7 Temporal data

A `2022 = 35%` and `2024 = 52%` are not contradictions — they're different points in time. Require an `as_of_date` or `publication_date` field on every data point that can change. Without it, growth-over-time reads as contradiction.

For facts that change (current CEO, current pricing), require the agent to **retrieve them fresh** from an authoritative tool rather than relying on training-time knowledge.

5.8 Re-grounding vs case-facts block

Both keep facts current across long sessions:

- **Case-facts block** — cheap, keeps facts the agent already verified.
- **Re-grounding** — re-reads tool results because the underlying facts may have changed (order status moved, customer record updated).

Use re-grounding when facts may have changed; use case-facts block otherwise.

5.9 Escalation triggers

Appropriate escalation reasons:

1. **Customer explicitly requests a human.** Honor immediately, don't first try to investigate.
2. **Policy is ambiguous or silent** on the customer's specific request (e.g., competitor price matching when policy only addresses own-site adjustments).
3. **Inability to make meaningful progress** after good-faith attempts.

Inappropriate escalation triggers:

- **Sentiment-based.** Customer frustration ≠ case complexity.
- **Self-reported confidence below threshold.** LLM confidence is uncalibrated.
- **Multiple matches on a customer search.** Ask for additional identifiers; don't pick heuristically and don't escalate.

When the customer is frustrated but the issue is within capability: acknowledge frustration, offer resolution, escalate only if they reiterate the preference for a human.

5.10 Data isolation

Per-customer scoping should be enforced **at the data layer**, not in prompts. The agent only receives facts about the current `customer_id`, enforced by the tool/service. Prompt rules ("don't bleed across customers") fail.

5.11 Confidence calibration

For high-volume extraction pipelines:

- Use **stratified random sampling** of high-confidence extractions to measure error rates and detect novel error patterns.
- Validate accuracy by **document type and field segment** — a 97% aggregate may mask 60% on a specific document type.
- Calibrate field-level confidence using a **labeled validation set** before using it for review routing.
- Route low-confidence and ambiguous-source extractions to humans first (limited reviewer capacity).

5.12 Multi-issue context

Three separate billing issues in one session: extract structured per-issue data (order IDs, amounts, statuses) into a separate context layer. Don't let them mix.

For multi-region agents (US, EU, AP), inject a `region` field in a structured session-context block; express policy as region-keyed rules. The agent doesn't have to remember the region; the structure carries it.

5.13 Internal consistency

When a synthesis output quotes different numbers in adjacent paragraphs: **first emit a structured fact table**, then have the narrative cite entries from that table. Inconsistency becomes detectable (and prevented).

PART 6 — The Recurring Patterns (the exam's grammar)

After studying every domain, you start seeing the same shapes recur. These are the meta-patterns.

6.1 The hierarchy of enforcement strength (revisited)

Weakest → strongest:

1. Stronger system prompt language — **almost always wrong**
2. Few-shot examples — useful for ambiguity, not enforcement
3. Tool description — useful for selection, not for sequencing or policy
4. JSON schema constraint — eliminates classes of errors
5. `tool_choice` configuration — eliminates "didn't call a tool"
6. `PostToolUse` hook — deterministic data normalization, output trimming
7. `PreToolUse` hook — deterministic blocking, prerequisites, policy
8. Tool-layer logic — input validation, idempotency, two-phase patterns

When a question describes "X happens N% of the time" and asks for the fix, climb the ladder. The right answer is usually 4 or higher.

6.2 "Address the root cause, not the symptom"

The exam writers love this. Common symptom-vs-root-cause pairs:

Symptom (wrong answer)	Root cause (right answer)
"Output has wrong format"	Use `tool_use` with a schema
"Agent skips a required step"	`PreToolUse` hook to enforce the prerequisite
"Agent fabricates fields"	Make the field nullable in the schema
"Agent picks the wrong tool"	Improve tool description specificity, then trim tool count
"Reports miss subtopics"	Coordinator decomposition was too narrow
"Subagents return shallow results"	They don't inherit context — pass it explicitly
"Long PR review is inconsistent"	Per-file passes + integration pass (attention dilution)
"Agent confuses dates between systems"	`PostToolUse` hook normalizes timestamps
"Agent re-creates duplicate tickets"	Require search before create + idempotency
"Conflicting numbers slip through"	Paired fields (stated_total + calculated_total)
"Sources lost during synthesis"	Required source field at every schema layer
"Context bloat across long sessions"	Trim at tool source + case-facts block + scratchpad

6.3 The distractor language patterns

The bank uses templated phrases on wrong answers. Recognizing them is half the battle:

- **"It's a commonly proposed patch that reduces the surface symptom without fixing the root cause."** → it's a symptom-fix
- **"The tempting shortcut; works until you hit an edge case that makes the real failure visible."** → looks easy, breaks under stress
- **"Works in the easy case and leaves the hard cases silently broken."** → over-narrow rule
- **"Buys short-term quiet at the cost of hiding a bug that will resurface under load."** → masks the problem
- **"Treats a reliability concern as a prompting concern, which is exactly the framing the guidance cautions against."** → prompt patch on a structural problem
- **"Looks plausible at first glance but creates its own failure modes you'd later have to work around."** → introduces new problems
- **"Feels rigorous but doesn't give the agent the information it needs to recover well."** → hides information from downstream
- **"It feels like the familiar lever, but it doesn't address the underlying structural issue."** → reaching for a comfortable tool that doesn't fit
- **"It is superficially attractive and ignores the layer where the actual control point lives."** → wrong layer
- **"Sounds safe, but the guidance explicitly points elsewhere for this shape of problem."** → there's a documented pattern, this isn't it

When you see one of these phrases on an option, that option is wrong. Use that to narrow the field quickly.

6.4 The "use a bigger model / window" anti-pattern

Almost always wrong. Means the answerer doesn't understand the actual mechanism. Bigger doesn't fix:

- Attention quality / lost-in-the-middle
- Probabilistic vs deterministic enforcement
- Vague criteria
- Stale tool results
- Coordinator decomposition
- Tool selection ambiguity

6.5 The "tell the model to be careful" anti-pattern

Always wrong. The model isn't a person; it doesn't have careful and uncareful modes. The fix is structural, not motivational.

PART 7 — The Four Misses From Your Practice Exam

Cementing what tripped you up specifically.

D1-001 — The agentic loop should not check for assistant text

You picked A ("system prompt doesn't instruct"). Correct: B ("checking for assistant text content is unreliable").

Why: The model can produce explanatory text *between* tool calls. Text content is not a termination signal; `stop_reason == "end_turn"` is. (See §0.5 and §1.1.1.)

D3-026 — Resuming a stale session

You picked C (re-read every file). Correct: A (start a new session and inject a structured summary).

Why: Resumption propagates stale tool results from prior turns. The reasoning was built on those results. Re-reading current files doesn't undo the model's prior conclusions. Throw the session out; carry forward only a clean summary. (See §1.6.2.)

D3-028 — Skill prompting for arguments

You picked D (`require_args: true`). Correct: C (`argument-hint: "..."`).

Why: `require_args` and `interactive: true` don't exist in SKILL.md frontmatter. The real mechanism is `argument-hint` — Claude Code uses the hint to prompt the user when the skill is invoked without input. The three SKILL.md frontmatter knobs to memorize are `context: fork` , `allowed-tools` , `argument-hint` . (See §3.4.1.)

D4-033 — Batch traceability

You picked A (the index in the batch). Correct: C (`custom_id` on each request).

Why: Batch ordering is not a guaranteed correlation mechanism. `custom_id` is the **designed** correlation field — it's carried back on responses unchanged. Set it to the user_id (or a concat you can invert) and correlation is trivial. (See §4.7.)

PART 8 — Quick-Reference Cheat Sheets

8.1 Files & locations

Path	Scope	Purpose
`~/.claude/CLAUDE.md`	User	Personal preferences, never shared
`<project>/CLAUDE.md` or `.claude/CLAUDE.md`	Project	Team standards, version-controlled
`<subdir>/CLAUDE.md`	Directory	Directory-specific conventions
`.claude/rules/<topic>.md`	Project	Conditional rules with YAML `paths` glob frontmatter
`.claude/commands/<name>.md`	Project	Project slash commands
`~/.claude/commands/<name>.md`	User	Personal slash commands
`.claude/skills/<name>/SKILL.md`	Project	Project skills with frontmatter
`~/.claude/skills/<name>/SKILL.md`	User	Personal skills
`<project>/.mcp.json`	Project	Shared MCP server config
`~/.claude.json`	User	Personal MCP servers

8.2 SKILL.md frontmatter

Knob	Effect
`context: fork`	Run in isolated sub-agent; only final output reaches main session
`allowed-tools: [...]`	Restrict tool access
`argument-hint: "..."`	Prompt user when invoked without args

(No `require_args`, no `interactive: true`, no `quiet: true`, no `return_only`, no `max_tokens`, no `readonly`.)

8.3 Claude Code CLI flags for CI

Flag	Purpose
`-p` / `--print`	Non-interactive mode (mandatory in CI)
`--output-format json`	JSON output
`--json-schema <file>`	Enforce a schema on the JSON output

Exit code: standard POSIX, non-zero on failure.

(No `--batch`, `--json-only`, `CLAUDE_HEADLESS`.)

8.4 `tool_choice` values

Value	Meaning
`"auto"`	Model may call a tool or return text (default)
`"any"`	Model must call some tool, picks which
`{"type": "tool", "name": "X"}`	Model must call tool X specifically

(No `"required"`.)

8.5 `stop_reason` values

Value	Loop action
`tool_use`	Execute, append result, loop
`end_turn`	Done, return to user
`max_tokens`	Handle (summarize partial, surface truncation)

8.6 Hooks

Hook	When	Used for
`PreToolUse`	Before tool executes	Block, modify, prerequisite gate, policy
`PostToolUse`	After tool returns	Normalize, trim, log, chain

8.7 Session management

Lever	When
`--resume <name>`	Continue specific session, prior context still valid
`fork_session`	Branch from shared baseline, compare independent paths
`/compact`	Reduce context, **facts still valid**
Start fresh + summary	**Facts stale** (code changed, world changed)

8.8 Tool error metadata

Field	Purpose
`isError: true`	MCP standard flag
`errorCategory`	`transient` / `validation` / `permission` / `business`
`isRetryable`	Boolean, **explicit**
Human-readable message	For agent communication

8.9 Schema design defaults

- Field may be absent in source → **nullable** (prevents fabrication)
- Categorical with possible new values → enum **+ "other"** + nullable detail string
- Cross-field semantic check needed → **paired fields** (stated_X + calculated_X)
- Uncertainty is a valid answer → enum **including** `"insufficient_evidence"` or `"unclear"`
- Money → `number`, separate `currency` field
- Time-varying data → require `as_of_date` / `publication_date`
- Provenance required → required `{name, url, publication_date}` with URL pattern

8.10 Decision: plan mode vs direct

Plan	Direct
Multi-file, multi-service, architectural decisions	Single file, well-scoped, clear fix
Multiple valid approaches	Decided approach
Library migration affecting 45+ files	Add a date-validation conditional
Microservices restructuring	Fix from a clear stack trace

8.11 Decision: batch API vs synchronous

Batch	Synchronous
Overnight reports	Pre-merge checks
Weekly audits	Interactive UX
Latency-tolerant (≤ 24h)	Blocking workflows
Single-turn extraction	Multi-turn tool calling
Cost-sensitive (50% savings)	Latency-sensitive

8.12 Decision: which Claude Code primitive

Need	Use
Always-on team standards	Project CLAUDE.md
Team standards by file type/pattern	`.claude/rules/` with glob `paths`
Team standards by directory	Subdirectory CLAUDE.md
Personal preferences	`~/.claude/CLAUDE.md`
Reusable team workflow	Project slash command or skill
Verbose-output skill	Skill with `context: fork`
Safe read-only skill	Skill with restricted `allowed-tools`

Conditional rules in CI	`.claude/rules/` with `paths`
Shared backend integration	`.mcp.json` with env-var secrets
Personal experimental tool	`~/.claude.json`

PART 9 — Final mental model

If everything in this guide collapsed into one sentence, it would be:

> ***The model is probabilistic. The system around the model is what makes it reliable. When in doubt, push the rule outward — into a hook, a schema, a tool-layer check, a structured field — until failure becomes impossible by construction.***

Every domain is a corollary:

- **Domain 1 (Agentic Architecture)** — the loop must be driven by `stop_reason`; deterministic enforcement (hooks, prerequisites) beats prompt instructions; coordinator-routed multi-agent systems with explicit context passing and parallel `Task` calls.
- **Domain 2 (Tools & MCP)** — tool descriptions drive selection; structured errors drive recovery; tool counts and naming drive accuracy; secrets via env vars; resources for ambient visibility.
- **Domain 3 (Claude Code)** — three-level CLAUDE.md hierarchy that stacks; `.claude/rules/` for path-conditional rules; skills for verbose isolated work; plan mode earned by ambiguity; CI via `-p` + JSON schema.
- **Domain 4 (Prompts & Schemas)** — explicit criteria over vague qualifiers; few-shot for ambiguity; `tool_use` + JSON schema for guaranteed structure; nullable for "may be absent"; paired fields for self-consistency; `custom_id` for batch correlation.
- **Domain 5 (Context & Reliability)** — case-facts blocks, scratchpads, source-side tool trimming; provenance as a required schema field at every layer; conflicts annotated not resolved silently; temporal data with explicit dates; lost-in-the-middle mitigated by structure and chunking.

Read this document twice. Then take the practice exams. Then re-read the sections on whatever you miss.

Good luck.